

Modelling Physical Climate Risk in IFRS 9 ECL Frameworks

A Plain-Language Guide to the ECL Prototype

*Two-Regime Markov-Switching Approach to Credit Migration — Zimbabwe's
Microfinance Sector*

Harare Institute of Technology (HIT) — Department of Applied Mathematics —
March 2026

Contents

1	What Is This Project?	3
1.1	The Problem in Plain English	3
1.2	Why It Matters	3
1.3	The Answer: A Two-Regime Switching Model	3
2	The Dataset	4
2.1	File Location	4
2.2	What Each Column Means	4
2.3	Key Numbers in the Dataset	4
3	How the System Works — Stage by Stage	5
3.1	Stage 1 — Data Loading (src/ingestion.py)	5
3.2	Stage 2 — Transition Extraction (src/transitions.py)	5
3.3	Stage 3 — Model Estimation (src/estimation.py)	6
3.4	Stage 4 — ECL Forecasting (src/forecasting.py)	6
3.5	Stage 5 — Backtesting (src/validation.py)	7
4	The Web Application — AgriFin Risk Monitor	8
4.1	Landing Page (/)	8
4.2	Farmer Self-Registration (/auth/signup)	8
4.3	Farmer Onboarding (/borrower/onboarding)	8
4.4	Farmer Dashboard (/borrower/dashboard)	9
4.5	Farmer Field Page (/borrower/field)	9
4.6	Farmer Report (/borrower/report)	9
4.7	Admin Dashboard (/admin/dashboard)	10
4.8	Admin Portfolio Page (/admin/portfolio)	10
4.9	Admin Borrower Detail (/admin/borrowers/[id])	10
4.10	Admin Field View (/admin/fields/[fieldId]/analyse)	10
4.11	Admin Loan Management	11
4.12	Admin Borrower Report (/admin/borrowers/[id]/report)	11
5	Process Tests	12
5.1	What the Tests Are For	12
5.2	Test Files and What They Cover	12
5.3	How to Run the Tests	12

5.4	Test Code Listings	14
5.4.1	Stage 1 — <code>tests/test_ingestion.py</code>	14
5.4.2	Stage 2 — <code>tests/test_transitions.py</code>	15
5.4.3	Stage 3 — <code>tests/test_estimation.py</code>	17
5.4.4	Stage 4 — <code>tests/test_forecasting.py</code>	18
5.4.5	Stage 5 — <code>tests/test_validation.py</code>	20
6	Quick Reference	23

What Is This Project?

The Problem in Plain English

Microfinance lenders in Zimbabwe give small loans to people who farm or run informal businesses. When a drought comes, borrowers struggle to repay — their crops fail, their income falls, and they miss payments. Under the international accounting rule IFRS 9, lenders must set aside money *before* losses happen (a “provision”). The amount set aside is called the **Expected Credit Loss (ECL)**.

The usual way of calculating ECL ignores whether it is drought season or not. It uses one fixed set of probabilities for borrowers moving from one credit rating to another. This project builds a smarter model that has *two sets* of probabilities — one for normal weather and one for drought — and switches between them depending on a drought index.

Why It Matters

- If ECL is underestimated during a drought, a lender could run out of capital and collapse.
- If ECL is overestimated every quarter, the lender holds back too much money and cannot lend to the next borrower.
- A climate-aware model gives a fairer, more accurate number.

The Answer: A Two-Regime Switching Model

Think of it like a thermostat that switches between two modes:

Condition	Mode	Which matrix is used?
Drought ($\gamma < -1.0$)	Stress	M_{stress} (higher default risk)
Normal ($\gamma \geq -1.0$)	Normal	M_{normal} (lower default risk)

In reality the model uses a smooth blend of both matrices, not a hard switch, so there is no abrupt jump at the threshold. The blending formula is:

$$P(\gamma) = \omega(\gamma) \cdot M_{\text{stress}} + (1 - \omega(\gamma)) \cdot M_{\text{normal}}$$

where $\omega(\gamma)$ is a number between 0 and 1 that gets closer to 1 the deeper the drought is.

The Dataset

File Location

```
jay_dataset.csv
```

What Each Column Means

Column	Meaning
borrower_id	Unique identifier for each borrower (e.g. MF01234).
loan_id	Unique identifier for the loan.
rating	Credit rating at that point in time. 1 = default (worst), 5 = best.
obs_date	The date this observation was recorded (one record per quarter per borrower).
district	The district the borrower lives in.
gamma	The SPEI-3 drought index for that period. Negative means dry; positive means wet. Below -1.0 is considered a drought.
default_flag	1 if the borrower is in default this quarter, 0 otherwise.
loan_amount	Outstanding loan balance in US dollars.

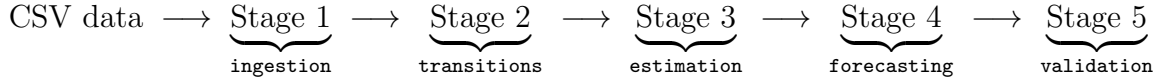
Key Numbers in the Dataset

- **6 borrowers**, observed over roughly 10 quarters each.
- **59 total observations**, producing **51 valid transitions** (pairs of consecutive quarterly ratings for the same borrower).
- **31 normal-regime** and **20 stress-regime** transitions.
- **5 credit states**: 1 (default/absorbing), 2, 3, 4, 5.
- Drought index γ ranges from about -1.84 to $+0.84$.

What is SPEI? The Standardised Precipitation Evapotranspiration Index (SPEI) measures how dry or wet conditions are compared with the long-term average. A value of 0 is normal; -1 means moderately dry; -2 means severely dry; $+1$ means unusually wet. The 3-month accumulation (SPEI-3) is used as the drought proxy γ throughout this project.

How the System Works — Stage by Stage

The prototype is built from five Python modules, each responsible for one stage of the pipeline.



Stage 1 — Data Loading (src/ingestion.py)

What it does in plain English: Reads the CSV file from disk, checks that all required columns are present, parses dates, makes sure ratings are between 1 and 5, and sorts everything in chronological order. If anything is wrong (file not found, missing column, bad rating value) it raises an error immediately before any calculation starts.

Key functions:

- `load_dataset(path)` — opens the file, parses and cleans it.
- `validate_dataset(df)` — checks columns and rating range.
- `dataset_summary(df)` — returns a dictionary of headline statistics (number of borrowers, date range, etc.).

Stage 2 — Transition Extraction (src/transitions.py)

What it does in plain English: For each borrower, looks at pairs of consecutive quarterly records and records what rating they moved from and to. For example, if a borrower was rated 3 in March and then rated 2 in June, that is a “3 → 2” transition. It then labels each transition as *normal* or *stress* depending on the SPEI value at that time.

Important rule: Transitions *from* State 1 (default) are excluded. Once a borrower defaults, they stay in default — there is nothing to model.

Key functions:

- `extract_rating_transitions(df)` — produces the transitions table.
- `build_count_matrix(transitions, regime)` — counts how many times each $i \rightarrow j$ transition was observed. Can be filtered to just normal or just stress transitions.
- `mle_matrix_from_counts(counts)` — converts counts into probabilities. If a rating state was never observed (e.g. State 5 had no transitions), a sensible “prior” probability is used so no row is left as all zeros.

Example count matrix (normal regime):

$$C_{\text{normal}} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 5 & 0 & 0 & 0 \\ 0 & 0 & 8 & 3 & 0 \\ 0 & 0 & 1 & 6 & 2 \\ 0 & 0 & 0 & 1 & 2 \end{pmatrix}$$

Row i , column j = number of times a borrower moved from State i to State j during normal conditions.

Stage 3 — Model Estimation (src/estimation.py)

What it does in plain English: Estimates the two matrices (M_{normal} and M_{stress}) and the two blending parameters (κ and γ_0) from the data.

Two-stage approach (why not do everything at once?) With only 51 transitions and 34 parameters, fitting everything simultaneously is unstable — the computer can reach wild numbers like $\kappa = 9 \times 10^{117}$. Instead:

1. **Stage 3a — Matrix estimation:** Use the hard-threshold labels (normal/stress) to compute M_{normal} and M_{stress} directly from the count matrices.
2. **Stage 3b — Scalar optimisation:** With the matrices fixed, find the values of κ and γ_0 that make the observed transitions most probable (maximum likelihood over just two numbers).

What κ and γ_0 mean:

- γ_0 is the “tipping point” — the SPEI level at which the model considers conditions to have switched from normal to drought. Estimated at -1.10 from the data.
- κ controls how sharply the switch happens. Estimated at 1.25 . A large κ means almost a hard switch; a small κ means a very gradual blend.

The regime weight formula:

$$\omega(\gamma) = \frac{1}{1 + e^{\kappa(\gamma - \gamma_0)}}$$

This is a decreasing S-curve: when $\gamma \ll \gamma_0$ (deep drought), $\omega \rightarrow 1$ (full stress weight); when $\gamma \gg \gamma_0$ (wet), $\omega \rightarrow 0$ (full normal weight).

Testing the two-regime model (LRT): The Likelihood Ratio Test asks: “does adding the second regime actually help?” It compares the log-likelihood of the two-regime model to a simpler single-regime model. A small p-value (below 0.05) would mean the two-regime model is significantly better. With only 51 transitions the test lacks power, but the structural distance metric (SVDM) still shows meaningful separation.

SVDM (Singular Value Decomposition Metric): A number that measures how different M_{normal} and M_{stress} are from each other. An SVDM of 0 means identical matrices (no regime effect); a large SVDM means the two regimes behave very differently.

Stage 4 — ECL Forecasting (src/forecasting.py)

What it does in plain English: For each active borrower, projects how likely they are to default over the next 5 years (20 quarters) and discounts those losses back to today.

Step by step:

1. **Pick the blended matrix** for the current climate (γ): $P(\gamma) = \omega \cdot M_{\text{stress}} + (1 - \omega) \cdot M_{\text{normal}}$.
2. **Multiply the matrix by itself h times** (P^h) to get the probability of being in each state after h quarters.
3. **Read off the default column** to get the cumulative probability of having defaulted by quarter h .

4. **Calculate the marginal (period-by-period) default probability:** how much did the default probability increase in quarter h compared to quarter $h - 1$?
5. **Apply the ECL formula** to each quarter and sum:

$$\text{ECL} = \sum_{h=1}^H \underbrace{(1+r)^{-h/4}}_{\text{discount factor}} \times \underbrace{\text{EAD}}_{\text{loan balance}} \times \underbrace{\text{LGD}}_{\text{loss fraction}} \times \underbrace{\text{PD}_h}_{\text{marginal default prob}}$$

where $r = 5\%$ per year and $\text{LGD} = 45\%$ (standard assumption).

Scenario analysis: The same calculation is repeated four times, using different fixed values of γ to represent different climate futures:

Scenario	γ override	Probability weight
Baseline	-0.73	55%
Moderate Drought	-1.20	25%
Severe Drought	-1.80	12%
Wet / Recovery	+0.80	8%

The probability-weighted average across all four scenarios gives the *expected ECL* that IFRS 9 requires.

Stage 5 — Backtesting ([src/validation.py](#))

What it does in plain English: Checks how well the model predicted what actually happened, using the same data it was trained on (in-sample check, because the dataset is too small for a true out-of-sample test).

Metrics:

- **AUC (Area Under the ROC Curve):** Measures discrimination — how well does the model separate borrowers who defaulted from those who did not?
 - 0.5 = no better than random.
 - 1.0 = perfect.
 - Values above 0.70 are considered acceptable in credit risk.
- **Brier Score:** Measures calibration — are the predicted probabilities numerically accurate? Think of it as the mean squared error of probabilities.
 - 0 = perfect.
 - Lower is better.

Both metrics are computed for the two-regime model and for a benchmark single-regime model so they can be compared side by side.

The Web Application — AgriFin Risk Monitor

The **AgriFin Risk Monitor** is a Next.js 16 web application that demonstrates the model in realistic operational use. It has two user roles — credit officer (Admin) and farmer (Borrower) — and replaces the earlier Streamlit prototype as the primary interactive deliverable.

The application runs at `http://localhost:3000` after starting the development server (see Section 6).

Landing Page (/)

The public home page explains the system in plain language without exposing any model parameters. It shows:

- The AgriFin brand name and purpose statement.
- A three-column feature section: satellite field analysis, two-regime credit migration, and IFRS 9 ECL forecasting.
- A model parameters strip showing $\kappa = 1.25$, $\gamma_0 = -1.10$, $\text{LGD} = 45\%$, and discount rate = 5%.
- Two call-to-action buttons: **Credit Officer Login** and **Register as a Farmer**.

Farmer Self-Registration (/auth/signup)

A farmer completes a registration form with name, email, password, phone, national ID, district, and primary activity. On success the account is created with role **BORROWER** and the farmer is redirected to onboarding.

Farmer Onboarding (/borrower/onboarding)

The first screen a new farmer sees after registration. It presents an interactive satellite map (Leaflet + ESRI World Imagery) with a polygon drawing tool. The farmer:

1. Names the field and selects a crop type.
2. Enters coordinates to fly the map to their location.
3. Draws the field boundary as a polygon.
4. Clicks **Save & Analyse**.

The system then:

1. Saves the field (area calculated via Shoelace formula, location reverse-geocoded via Nominatim).
2. Fetches 14-day temperature and rainfall from the Open-Meteo forecast API for the polygon centroid.
3. Fetches 36+ months of daily historical temperature and rainfall from the Open-Meteo archive API.

4. Computes SPEI-3 via the Thornthwaite PET method and an empirical cumulative distribution function — the same methodology as Vicente-Serrano et al. (2010).
5. Derives the agricultural health score (0–100) and agronomic recommendations.
6. Displays the SPEI value, WMO drought classification, and agri score.

Farmer Dashboard (/borrower/dashboard)

The farmer’s home screen after login. Four areas:

- **My Loan** — loan reference, credit standing (plain language), outstanding balance, loan status, and a 12-month credit exposure estimate if an ECL has been computed.
- **Drought Index (SPEI)** — the SPEI-3 value with WMO label (e.g. “Moderate Drought”) and a plain-language explanation. Shows which drought classification band applies using colour coding.
- **Farm Health** — agri score, NDVI, 14-day rainfall, temperature, and the date of last analysis.
- **Recommendations** — plain-language agronomic advice derived from the rule-based engine.

If no field has been set up, a banner prompts the farmer to complete onboarding.

Farmer Field Page (/borrower/field)

Detailed field view showing the polygon on a satellite map alongside SPEI, agri score, NDVI metrics, and recommendations. A **Re-analyse** button repeats the full SPEI and satellite data fetch for the same polygon, updating all stored values.

Farmer Report (/borrower/report)

A print-ready document with six sections:

1. **My Details** — name, national ID, district, activity, phone, member-since date.
2. **My Loan** — reference, credit standing, balance, status, disbursement date, number of payment records.
3. **Climate & Weather** (shown only when data exists) — SPEI-3 value, 14-day rainfall, average temperature, and a plain-language drought description. Hidden if no field analysis has been run.
4. **Farm Health** — agri score, NDVI, trend, rainfall, temperature, water-stress flag.
5. **Credit Exposure Estimate** — 12-month and lifetime ECL in plain language, with a disclaimer that these are IFRS 9 provisions, not amounts owed.
6. **Recommendations** — agronomic advice list.

No raw model parameters (κ , γ_0 , ω , PD) are shown to the farmer. A **Save as PDF** button triggers the browser print dialog.

Admin Dashboard (/admin/dashboard)

The credit officer's home screen. Four summary cards show total borrowers, total EAD, 1-year portfolio ECL, and number of defaulted loans. Below them is the active loan book table with one row per loan showing:

- Borrower name and district.
- Loan reference and current credit rating (1–5).
- Exposure at Default (EAD).
- Regime badge — Normal / Moderate Stress / Drought Stress, derived from $\omega(\gamma)$.
- One-quarter PD and 1-year ECL.
- IFRS 9 stage (Stage 1 / Stage 2 / Stage 3).

Navigation links to the Portfolio page and a **Register Borrower** button.

Admin Portfolio Page (/admin/portfolio)

Portfolio-wide risk analytics:

- ECL by IFRS 9 scenario (bar chart: baseline, moderate drought, severe drought, wet recovery) with probability weights and the expected ECL marked.
- Regime breakdown (how many loans are in Normal / Moderate / Stress regime).
- Rating distribution across the portfolio.
- Full loan table with γ , ω , PD, and ECL columns.

Admin Borrower Detail (/admin/borrowers/[id])

Three summary cards at the top:

- **Profile** — name, national ID, district, primary activity, phone, registration date.
- **Credit Risk (Latest)** — current rating, IFRS 9 stage, EAD, regime weight $\omega(\gamma)$, one-quarter PD, 1-year ECL, 5-year ECL.
- **Agricultural Signal** — SPEI-3 γ , agri score (0–100), mean NDVI, 14-day rainfall, average temperature. A **View Full Field Details** link opens the read-only field page. If the farmer has not yet set up their field, a plain message says so.

Below the cards: the loan history table and the rating observation timeline for the most recent loan.

Admin cannot create or analyse fields. Field boundaries and satellite analyses are created exclusively by the farmer through the self-service onboarding and re-analyse flows. The admin reads the data the farmer has produced — they cannot modify it.

Admin Field View (/admin/fields/[fieldId]/analyse)

A read-only view of the farmer's latest field analysis. Displays:

- Four metric cards: agri score, SPEI-3 γ (with regime weight ω), 14-day rainfall, and average temperature.
- Vegetation signal detail: mean NDVI, NDVI trend, health status, and analysis timestamp.
- ECL forecast: one-quarter PD, 1-year ECL, 5-year ECL, and the four IFRS 9 scenario breakdown.
- Agronomic recommendations.

If the farmer has not yet run an analysis the page shows a clear message rather than empty data.

Admin Loan Management

`/admin/loans/new` — Creates a new loan for a borrower. Fields: loan reference, initial credit rating, loan amount (EAD), disbursement date, and IFRS 9 stage. On save the ECL engine runs immediately using the borrower's most recent SPEI value.

`/admin/loans/[id]` — Loan detail page showing the ECL summary and the full rating observation history table.

`/admin/loans/[id]/add-observation` — Adds a quarterly rating observation: period label (e.g. 2024Q4), date, credit rating, default flag, γ (SPEI value), and outstanding balance. After saving the ECL engine re-runs for the updated γ .

Admin Borrower Report (`/admin/borrowers/[id]/report`)

A print-ready six-section IFRS 9 report for the credit officer:

1. Borrower and loan summary (profile fields, loan reference, rating, EAD, stage, disbursement date).
2. Climate and drought (SPEI-3 γ , WMO label, regime weight ω , drought stress interpretation).
3. Agricultural health (agri score, NDVI, trend, health status, rainfall, temperature, water-stress and disease-risk flags).
4. Credit risk metrics (one-quarter PD, 1-year ECL, 5-year ECL).
5. IFRS 9 scenario analysis (four scenario ECL values with probability weights, expected ECL).
6. Recommendations (agronomic advice from the rule-based engine).

Technical model parameters are shown on this report because it is intended for the credit officer, not the farmer.

Process Tests

What the Tests Are For

The tests verify that each stage of the pipeline produces correct output *before* the real dataset is touched. They use small, hand-crafted datasets so a failure points directly to the responsible function. Running the tests after any code change confirms nothing was broken.

All 82 tests run in under 3 seconds.

Test Files and What They Cover

File	What it tests
<code>test_ingestion.py</code>	File not found; missing columns; out-of-range ratings; correct types; sorted order; <code>dataset_summary</code> keys and counts.
<code>test_transitions.py</code>	Absorbing state excluded; regime labels; count matrix shape and dtype; regime filtering; row-stochastic MLE matrix; zero-count rows handled.
<code>test_estimation.py</code>	Regime weight at drought/wet/threshold; no overflow; array input; blend matrix row-stochastic; <code>estimate_model</code> converges with $\kappa > 0$; LRT returns finite statistic and valid p-value.
<code>test_forecasting.py</code>	Cumulative PD monotone and in $[0, 1]$; marginal PD non-negative; ECL positive and higher under drought; EAD scaling; portfolio total row; scenario count and expected row.
<code>test_validation.py</code>	AUC returns NaN for single class; perfect/-worst/random AUC; Brier score 0 for perfect; non-negative; ROC array lengths; backtest columns; <code>y_true</code> binary; <code>y_pred</code> in $[0, 1]$; validation table returns 2 rows.

How to Run the Tests

Step 1 — Open a terminal and navigate to the tests folder:

```
cd /path/to/physical-climate-risk/tests
```

Step 2 — Run all tests with uv:

```
uv run pytest tests/ -v
```

The `-v` flag shows each test name and PASSED / FAILED.

To run only one test file:

```
uv run pytest tests/test_ingestion.py -v
```

To run one specific test by name:

```
uv run pytest
    tests/test_estimation.py::TestRegimeWeight::test_deep_drought_approaches_c
-v
```

Expected output when all tests pass:

```
===== 82 passed, 8 warnings in 1.89s
=====
```

Note on the uv tool. This project uses `uv` to manage dependencies. It is a fast Python package manager. Running any command with `uv run ...` automatically uses the correct virtual environment and installed packages. You do not need to activate a virtual environment manually.

Test Code Listings

The complete source for each test file is shown below for reference.

5.4.1 Stage 1 — tests/test_ingestion.py

```

1  """
2  Stage 1 tests: data loading and validation.
3  """
4  import sys, os, unittest, tempfile
5  import pandas as pd
6
7  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
8  from src.ingestion import load_dataset, validate_dataset,
   dataset_summary
9
10 VALID_CSV = """\
11 borrower_id,loan_id,rating,obs_date,district,gamma,default_flag,loan_amount
12 B01,L01,3,2022-03-31,Harare,-0.50,0,5000
13 B01,L01,2,2022-06-30,Harare,-1.20,0,4800
14 B02,L02,4,2022-03-31,Bulawayo,0.30,0,8000
15 B02,L02,1,2022-06-30,Bulawayo,-1.50,1,0
16 """
17
18 def _write_temp_csv(content):
19     tmp = tempfile.NamedTemporaryFile(
20         mode="w", suffix=".csv", delete=False)
21     tmp.write(content); tmp.close()
22     return tmp.name
23
24 class TestLoadDataset(unittest.TestCase):
25
26     def test_file_not_found(self):
27         with self.assertRaises(FileNotFoundError):
28             load_dataset("/does/not/exist.csv")
29
30     def test_valid_csv_loads(self):
31         path = _write_temp_csv(VALID_CSV)
32         df = load_dataset(path)
33         self.assertEqual(len(df), 4)
34         os.unlink(path)
35
36     def test_sorted_by_borrower_and_date(self):
37         path = _write_temp_csv(VALID_CSV)
38         df = load_dataset(path)
39         for _, grp in df.groupby("borrower_id"):
40             dates = grp["obs_date"].tolist()
41             self.assertEqual(dates, sorted(dates))
42         os.unlink(path)
43
44 class TestValidateDataset(unittest.TestCase):
45

```

```

46     def test_missing_column_raises(self):
47         df = pd.DataFrame({"borrower_id": ["B01"],
48                             "rating": [3]})    # missing columns
49         with self.assertRaises(ValueError):
50             validate_dataset(df)
51
52     def test_out_of_range_rating_raises(self):
53         path = _write_temp_csv(
54             "borrower_id,loan_id,rating,obs_date,"
55             "district,gamma,default_flag,loan_amount\n"
56             "B01,L01,9,2022-03-31,Harare,-0.5,0,5000\n")
57         df = pd.read_csv(path)
58         df["obs_date"] = pd.to_datetime(df["obs_date"])
59         df["rating"] = pd.to_numeric(df["rating"])
60         df["gamma"] = pd.to_numeric(df["gamma"])
61         df["loan_amount"] = pd.to_numeric(df["loan_amount"])
62         df["default_flag"] = 0
63         with self.assertRaises(ValueError):
64             validate_dataset(df)
65         os.unlink(path)
66
67 class TestDatasetSummary(unittest.TestCase):
68
69     def test_summary_counts_match(self):
70         path = _write_temp_csv(VALID_CSV)
71         df = load_dataset(path)
72         s = dataset_summary(df)
73         self.assertEqual(s["Total observations"], 4)
74         self.assertEqual(s["Unique borrowers"], 2)
75         os.unlink(path)
76
77 if __name__ == "__main__":
78     unittest.main(verbosity=2)

```

Listing 1: test_ingestion.py — Stage 1: Data Loading and Validation

5.4.2 Stage 2 — tests/test_transitions.py

```

1  """
2  Stage 2 tests: transition extraction and count matrices.
3  """
4  import sys, os, unittest
5  import numpy as np, pandas as pd
6
7  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
8  from src.transitions import (
9      extract_rating_transitions, build_count_matrix,
10     mle_matrix_from_counts,
11 )
12 from config import N_STATES
13
14 def _make_panel():

```

```

15     rows = [
16         {"borrower_id": "B01", "rating": 3, "obs_date": pd.Timestamp("2022-03-31"),
17          "gamma": -0.5, "loan_amount": 5000, "default_flag": 0,
18          "district": "Harare", "loan_id": "L01"},
19         {"borrower_id": "B01", "rating": 2, "obs_date": pd.Timestamp("2022-06-30"),
20          "gamma": -1.5, "loan_amount": 4800, "default_flag": 0,
21          "district": "Harare", "loan_id": "L01"},
22         {"borrower_id": "B01", "rating": 1, "obs_date": pd.Timestamp("2022-09-30"),
23          "gamma": -1.8, "loan_amount": 0, "default_flag": 1,
24          "district": "Harare", "loan_id": "L01"},
25         {"borrower_id": "B02", "rating": 4, "obs_date": pd.Timestamp("2022-03-31"),
26          "gamma": 0.3, "loan_amount": 8000, "default_flag": 0,
27          "district": "Bulawayo", "loan_id": "L02"},
28         {"borrower_id": "B02", "rating": 3, "obs_date": pd.Timestamp("2022-06-30"),
29          "gamma": 0.2, "loan_amount": 7500, "default_flag": 0,
30          "district": "Bulawayo", "loan_id": "L02"},
31     ]
32     return pd.DataFrame(rows)
33
34 class TestExtractRatingTransitions(unittest.TestCase):
35
36     def setUp(self):
37         self.t = extract_rating_transitions(_make_panel())
38
39     def test_absorbing_state_excluded(self):
40         self.assertTrue((self.t["from_rating"] != 1).all())
41
42     def test_expected_transition_count(self):
43         # 3->2 (normal), 2->1 (stress), 4->3 (normal) = 3 total
44         self.assertEqual(len(self.t), 3)
45
46     def test_regime_labels_correct(self):
47         normal = self.t[self.t["gamma"] >= -1.0]
48         self.assertTrue((normal["regime"] == "normal").all())
49         stress = self.t[self.t["gamma"] < -1.0]
50         self.assertTrue((stress["regime"] == "stress").all())
51
52 class TestMleMatrixFromCounts(unittest.TestCase):
53
54     def test_row_stochastic(self):
55         counts = np.zeros((N_STATES, N_STATES), dtype=int)
56         counts[2, 1] = 1; counts[2, 2] = 1
57         P = mle_matrix_from_counts(counts)
58         np.testing.assert_allclose(
59             P.sum(axis=1), np.ones(N_STATES), atol=1e-9)
60
61     def test_absorbing_row(self):
62         counts = np.zeros((N_STATES, N_STATES), dtype=int)
63         P = mle_matrix_from_counts(counts)
64         expected = np.zeros(N_STATES); expected[0] = 1.0
65         np.testing.assert_allclose(P[0], expected, atol=1e-12)

```

```

66
67 if __name__ == "__main__":
68     unittest.main(verbosity=2)

```

Listing 2: test_transitions.py — Stage 2: Transition Extraction

5.4.3 Stage 3 — tests/test_estimation.py

```

1  """
2  Stage 3 tests: regime weight, model estimation, LRT.
3  """
4  import sys, os, unittest
5  import numpy as np, pandas as pd
6
7  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
8  from src.estimation import (
9      regime_weight, blend_matrices,
10     estimate_model, likelihood_ratio_test,
11 )
12 from config import N_STATES
13
14 def _make_transitions():
15     rows = [
16         {"borrower_id": "B01", "from_rating": 3, "to_rating": 2,
17          "gamma": -0.5, "regime": "normal"},
18         {"borrower_id": "B01", "from_rating": 2, "to_rating": 1,
19          "gamma": -1.5, "regime": "stress"},
20         {"borrower_id": "B02", "from_rating": 4, "to_rating": 3,
21          "gamma": 0.3, "regime": "normal"},
22         {"borrower_id": "B02", "from_rating": 3, "to_rating": 3,
23          "gamma": 0.1, "regime": "normal"},
24         {"borrower_id": "B03", "from_rating": 5, "to_rating": 4,
25          "gamma": -1.2, "regime": "stress"},
26         {"borrower_id": "B03", "from_rating": 4, "to_rating": 4,
27          "gamma": -0.8, "regime": "normal"},
28         {"borrower_id": "B04", "from_rating": 2, "to_rating": 2,
29          "gamma": -1.8, "regime": "stress"},
30         {"borrower_id": "B04", "from_rating": 3, "to_rating": 2,
31          "gamma": -2.0, "regime": "stress"},
32     ]
33     return pd.DataFrame(rows)
34
35 class TestRegimeWeight(unittest.TestCase):
36
37     def test_deep_drought_approaches_one(self):
38         w = regime_weight(-10.0, kappa=2.0, gamma0=-1.0)
39         self.assertGreater(w, 0.99)
40
41     def test_wet_conditions_approaches_zero(self):
42         w = regime_weight(10.0, kappa=2.0, gamma0=-1.0)
43         self.assertLess(w, 0.01)
44

```

```

45     def test_at_threshold_is_half(self):
46         w = regime_weight(-1.0, kappa=2.0, gamma0=-1.0)
47         self.assertEqual(w, 0.5, places=9)
48
49     def test_no_overflow_extreme_gamma(self):
50         w_low = regime_weight(-100.0, kappa=5.0, gamma0=-1.0)
51         w_high = regime_weight(100.0, kappa=5.0, gamma0=-1.0)
52         self.assertFalse(np.isnan(w_low) or np.isnan(w_high))
53
54 class TestEstimateModel(unittest.TestCase):
55
56     def setUp(self):
57         self.result = estimate_model(_make_transitions())
58
59     def test_kappa_positive(self):
60         self.assertGreater(self.result["kappa"], 0.0)
61
62     def test_matrices_row_stochastic(self):
63         for key in ["M_normal", "M_stress"]:
64             row_sums = self.result[key].sum(axis=1)
65             np.testing.assert_allclose(
66                 row_sums, np.ones(N_STATES), atol=1e-9)
67
68     def test_empty_transitions_raises(self):
69         with self.assertRaises(ValueError):
70             estimate_model(pd.DataFrame())
71
72 if __name__ == "__main__":
73     unittest.main(verbosity=2)

```

Listing 3: test_estimation.py — Stage 3: Model Estimation

5.4.4 Stage 4 — tests/test_forecasting.py

```

1  """
2  Stage 4 tests: cumulative PD, marginal PD, ECL, scenarios.
3  """
4  import sys, os, unittest
5  import numpy as np, pandas as pd
6
7  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
8  from src.forecasting import (
9      cumulative_pd, marginal_pd, compute_ecl,
10     portfolio_ecl, scenario_analysis,
11 )
12 from config import N_STATES, DEFAULT_HORIZON, SCENARIOS
13
14 def _P():
15     return np.array([
16         [1.00, 0.00, 0.00, 0.00, 0.00],
17         [0.10, 0.80, 0.07, 0.02, 0.01],
18         [0.04, 0.06, 0.78, 0.09, 0.03],

```

```

19         [0.01,0.02,0.06,0.82,0.09] ,
20         [0.00,0.01,0.03,0.08,0.88] ,
21     ])
22
23 def _Ps(): # stress (higher defaults)
24     return np.array([
25         [1.00,0.00,0.00,0.00,0.00] ,
26         [0.20,0.70,0.06,0.03,0.01] ,
27         [0.08,0.10,0.70,0.09,0.03] ,
28         [0.03,0.04,0.08,0.76,0.09] ,
29         [0.01,0.02,0.04,0.10,0.83] ,
30     ])
31
32 class TestCumulativePD(unittest.TestCase):
33
34     def test_non_decreasing(self):
35         cum = cumulative_pd(3, _P(), DEFAULT_HORIZON)
36         self.assertTrue(np.all(np.diff(cum) >= -1e-12))
37
38     def test_values_in_unit_interval(self):
39         cum = cumulative_pd(3, _P(), DEFAULT_HORIZON)
40         self.assertTrue(np.all(cum >= 0) and np.all(cum <=
41             1+1e-12))
42
43     def test_higher_risk_borrower_higher_pd(self):
44         cum2 = cumulative_pd(2, _P(), DEFAULT_HORIZON)
45         cum5 = cumulative_pd(5, _P(), DEFAULT_HORIZON)
46         self.assertGreater(cum2[-1], cum5[-1])
47
48 class TestComputeECL(unittest.TestCase):
49
50     def test_ecl_positive(self):
51         r = compute_ecl(3, 5000.0, -0.5, _P(), _Ps(), 2.0, -1.0)
52         self.assertGreater(r["ecl"], 0.0)
53
54     def test_ecl_higher_under_drought(self):
55         ecl_n =
56             compute_ecl(3,5000.0,-0.5,_P(),_Ps(),2.0,-1.0)["ecl"]
57         ecl_d =
58             compute_ecl(3,5000.0,-1.8,_P(),_Ps(),2.0,-1.0)["ecl"]
59         self.assertGreater(ecl_d, ecl_n)
60
61     def test_ead_scaling(self):
62         e1 =
63             compute_ecl(3,5000.0,-0.5,_P(),_Ps(),2.0,-1.0)["ecl"]
64         e2 =
65             compute_ecl(3,10000.0,-0.5,_P(),_Ps(),2.0,-1.0)["ecl"]
66         self.assertAlmostEqual(e2/e1, 2.0, places=6)
67
68 class TestPortfolioECL(unittest.TestCase):

```

```

65     def test_portfolio_total_row(self):
66         loans = pd.DataFrame([
67             {"borrower_id": "B01", "rating": 3, "loan_amount": 5000, "gamma": -0.5}
68             {"borrower_id": "B02", "rating": 4, "loan_amount": 8000, "gamma": -0.5}
69         ])
70         df = portfolio_ecl(loans, _P(), _Ps(), 2.0, -1.0)
71         self.assertIn("PORTFOLIO TOTAL", df["Borrower
72             ID"].values)
73
74 if __name__ == "__main__":
75     unittest.main(verbosity=2)

```

Listing 4: test_forecasting.py — Stage 4: ECL Forecasting

5.4.5 Stage 5 — tests/test_validation.py

```

1  """
2  Stage 5 tests: AUC, Brier Score, backtest predictions.
3  """
4  import sys, os, unittest
5  import numpy as np, pandas as pd
6
7  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
8  from src.validation import (
9      calculate_auc, calculate_brier_score,
10     backtest_predictions, validation_metrics,
11 )
12 from src.transitions import build_count_matrix,
13     mle_matrix_from_counts
14
15 def _P(dp=0.05):
16     K = 5; P = np.zeros((K,K))
17     P[0,0] = 1.0
18     for i in range(1,K):
19         P[i,0] = dp; P[i,i] = 1-dp
20     return P
21
22 def _transitions():
23     rows = [
24         {"borrower_id": "B01", "from_rating": 2, "to_rating": 1,
25         "gamma": -1.5, "regime": "stress"},
26         {"borrower_id": "B02", "from_rating": 3, "to_rating": 1,
27         "gamma": -1.8, "regime": "stress"},
28         {"borrower_id": "B03", "from_rating": 4, "to_rating": 4,
29         "gamma": -0.5, "regime": "normal"},
30         {"borrower_id": "B04", "from_rating": 5, "to_rating": 4,
31         "gamma": 0.3, "regime": "normal"},
32         {"borrower_id": "B05", "from_rating": 3, "to_rating": 3,
33         "gamma": -0.2, "regime": "normal"},
34         {"borrower_id": "B06", "from_rating": 2, "to_rating": 2,
35         "gamma": -1.2, "regime": "stress"},
36     ]

```

```

36     return pd.DataFrame(rows)
37
38 class TestCalculateAUC(unittest.TestCase):
39
40     def test_single_class_returns_nan(self):
41         result = calculate_auc(np.array([0,0,0]),
42                                np.array([0.1,0.2,0.3]))
43         self.assertTrue(np.isnan(result))
44
45     def test_perfect_returns_one(self):
46         result = calculate_auc(np.array([1,1,0,0]),
47                                np.array([0.9,0.8,0.1,0.2]))
48         self.assertAlmostEqual(result, 1.0, places=6)
49
50 class TestCalculateBrierScore(unittest.TestCase):
51
52     def test_perfect_returns_zero(self):
53         result = calculate_brier_score(np.array([1,0]),
54                                        np.array([1.0,0.0]))
55         self.assertAlmostEqual(result, 0.0, places=9)
56
57     def test_non_negative(self):
58         result = calculate_brier_score(np.array([1,0,0]),
59                                        np.array([0.6,0.3,0.1]))
60         self.assertGreaterEqual(result, 0.0)
61
62 class TestBacktestPredictions(unittest.TestCase):
63
64     def test_y_true_binary(self):
65         df = backtest_predictions(
66             _transitions(), _P(0.04), _P(0.12), 2.0, -1.0)
67         self.assertTrue(set(df["y_true"].unique()).issubset({0,1}))
68
69     def test_y_pred_in_unit_interval(self):
70         df = backtest_predictions(
71             _transitions(), _P(0.04), _P(0.12), 2.0, -1.0)
72         self.assertTrue((df["y_pred"] >= 0).all()
73                         and (df["y_pred"] <= 1).all())
74
75 class TestValidationMetrics(unittest.TestCase):
76
77     def test_returns_two_rows(self):
78         t = _transitions()
79         M_single = mle_matrix_from_counts(
80             build_count_matrix(t, regime=None))
81         df = validation_metrics(
82             t, _P(0.04), _P(0.12), 2.0, -1.0, M_single)
83         self.assertEqual(len(df), 2)
84
85 if __name__ == "__main__":
86     unittest.main(verbosity=2)

```

Listing 5: test_validation.py — Stage 5: Backtesting Metrics

Quick Reference

Task	Command
Run all pipeline tests	<code>cd tests && uv run pytest tests/ -v</code>
Run one test file	<code>uv run pytest tests/test_ingestion.py -v</code>
Start the web app	<code>cd application && pnpm dev</code>
Seed admin + matrices	<code>pnpm tsx prisma/seed.ts</code>
Run DB migrations	<code>pnpm prisma migrate dev</code>
Compile this PDF	<code>pdflatex explanation.tex</code>
Compile the dissertation	<code>pdflatex document/document.tex</code>

Project file structure summary:

physical-climate-risk/
 jay_dataset.csv — MFI panel dataset (6 borrowers, 59 obs)
 tests/ — Python ECL pipeline
 src/ingestion.py — Stage 1: data loading
 src/transitions.py — Stage 2: transition extraction
 src/estimation.py — Stage 3: model estimation
 src/forecasting.py — Stage 4: ECL calculation
 src/validation.py — Stage 5: backtesting
 tests/ — Unit tests (one file per stage, 82 total)
 application/ — Next.js web prototype (AgriFin Risk Monitor)
 app/admin/ — Credit officer pages (dashboard, borrowers, loans, portfolio)
 app/borrower/ — Farmer pages (dashboard, onboarding, field, report)
 lib/ecl/engine.ts — Two-Regime ECL computation (TypeScript)
 lib/gee/spei.ts — SPEI-3 implementation (Open-Meteo archive)
 prisma/schema.prisma — Database schema
 prisma/seed.ts — Admin account + migration matrices
 APP_STRUCTURE.md — Full route map and architecture notes
 document/document.tex — Full research dissertation (LaTeX)
 explanation/explanation.tex — This document